Part 2 — HDFS Administration

NYC taxi dataset on the Kind cluster. Commands, screenshots, and what happened.

Cluster: hadoop-lab Namespace: hadoop NameNode pod: namenode-0

Dataset path in HDFS: /datasets/nyc-taxi

I did this on the same HDFS from Part 1. Part 1 was running with 2 DataNodes and dfs.replication=2. For this part the assignment wants replication factor 3, so I added a third DataNode first. A block is stored once per DataNode. If the factor is 3, HDFS needs 3 different DataNodes or the extra copies can never be placed.

Why replication factor and DataNode count have to match

Replication factor is how many copies of each block HDFS keeps. Those copies must sit on different DataNodes. HDFS will not put two copies of the same block on one node.

In Part 1 I had 2 DataNodes, so the highest factor that can actually be satisfied is 2. That is why hdfs-site.xml still has dfs.replication=2 as the cluster default. fsck later prints “Default replication factor: 2”. That line is the cluster default. It is not the factor of the taxi files. I changed only /datasets/nyc-taxi to 3 with hdfs dfs -setrep.

For factor 3 I scaled the DataNode StatefulSet from 2 to 3. datanode-2 crashed once on a new volume because the directory was owned by root and the DataNode process runs as user hadoop (uid 1000). A small init container now does chown 1000:100 on the data volume before the DataNode starts. After that, dfsadmin -report showed Live datanodes (3).

What factor 4 or 5 would need

Same rule. The factor cannot be higher than the number of live DataNodes.

- RF=2 needs at least 2 DataNodes. That is what Part 1 used.
- RF=3 needs at least 3 DataNodes. That is what this part uses.
- RF=4 needs at least 4 DataNodes. With only 3, fsck would keep showing under-replicated blocks. The 4th copy has nowhere to go.
- RF=5 needs at least 5 DataNodes. With 3 DataNodes, 2 copies of every block would stay missing until two more DataNodes joined.
- If I had 5 DataNodes and RF=3, and I stopped one node, HDFS could copy the missing replica onto one of the spare nodes without waiting for the dead pod to come back. I did not add a 4th node. With exactly 3 nodes, the third copy comes back only after datanode-2 is running again.

1. Upload a 500 MB+ dataset

I used the public NYC TLC yellow taxi trip files for 2019, months February to June. They were already on the VM under /opt/hadoop-lab/hdfs/data/. I skipped January. That file on the VM was only 128 KB, so it was incomplete. February to June together are 518.7 MB, which is over the 500 MB the task asks for.

I copied each parquet file into the NameNode pod with kubectl cp, then put them into HDFS.

```
kubectl exec -n hadoop namenode-0 -- hdfs dfs -mkdir -p /datasets/nyc-taxi

kubectl cp /opt/hadoop-lab/hdfs/data/yellow_tripdata_2019-02.parquet hadoop/namenode-
0:/tmp/yellow_tripdata_2019-02.parquet
kubectl cp /opt/hadoop-lab/hdfs/data/yellow_tripdata_2019-03.parquet hadoop/namenode-
0:/tmp/yellow_tripdata_2019-03.parquet
kubectl cp /opt/hadoop-lab/hdfs/data/yellow_tripdata_2019-04.parquet hadoop/namenode-
0:/tmp/yellow_tripdata_2019-04.parquet
kubectl cp /opt/hadoop-lab/hdfs/data/yellow_tripdata_2019-05.parquet hadoop/namenode-
0:/tmp/yellow_tripdata_2019-05.parquet
kubectl cp /opt/hadoop-lab/hdfs/data/yellow_tripdata_2019-06.parquet hadoop/namenode-
0:/tmp/yellow_tripdata_2019-06.parquet

kubectl exec -n hadoop namenode-0 -- bash -lc 'hdfs dfs -put -f /tmp/yellow_tripdata_2019-
0{2,3,4,5,6}.parquet /datasets/nyc-taxi/'
kubectl exec -n hadoop namenode-0 -- hdfs dfs -du -h /datasets/nyc-taxi
kubectl exec -n hadoop namenode-0 -- hdfs dfs -count -h /datasets/nyc-taxi
kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/nyc-taxi
```

hdfs dfs -count printed 5 files and 518.7 M. hdfs dfs -ls showed the second column as 2. That column is the replication of each file. At this moment the files still had the cluster default of 2. The put itself worked. The “No such file” on /tmp inside the pod is only because -put moved the temp copies. The files are in HDFS.

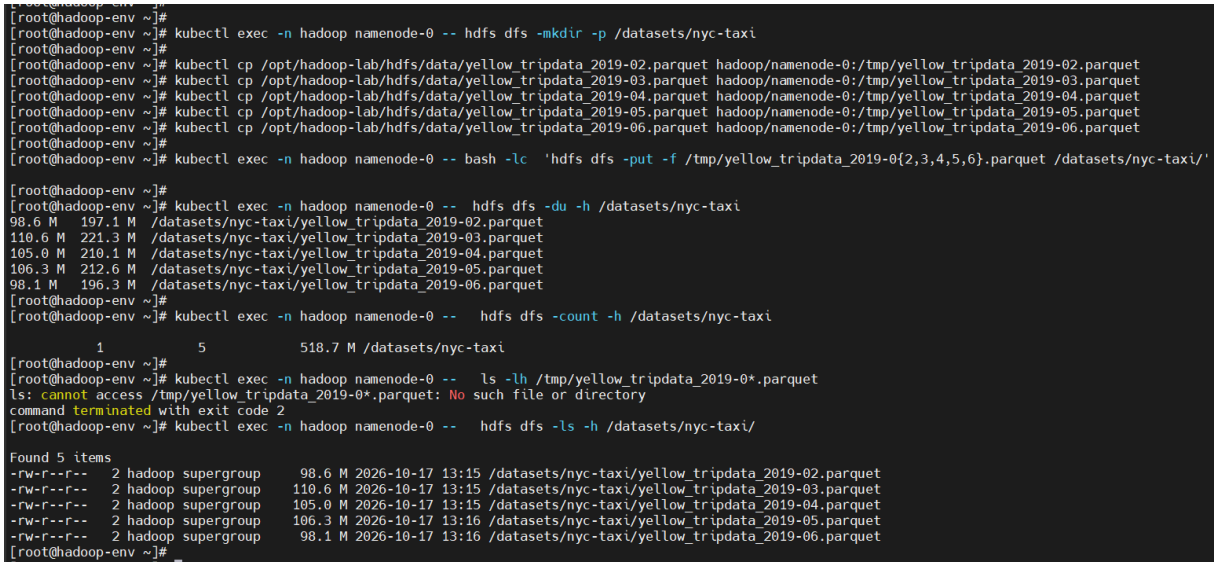


Figure 1. Upload of Feb–Jun taxi files. Total 518.7 MB. Replication column is still 2.

2. Set replication factor to 3 and check with fsck

I changed only this directory, not the cluster-wide default.

```
kubectl exec -n hadoop namenode-0 -- hdfs dfs -setrep -R 3 /datasets/nyc-taxi
kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/nyc-taxi
kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks -locations
```

setrep printed “Replication 3 set” for all 5 files. ls then showed 3 in the replication column. fsck listed each file as replication=3, 1 block, Live_repl=3, and OK. Each block had three DatanodeInfoWithStorage lines, one on each DataNode (10.244.1.34, 10.244.2.37, and 10.244.1.35). One block per file is normal here because the default block size is 128 MB and each file is about 100 MB.

```
[root@hadoop-env ~]#
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -setrep -R 3 /datasets/nyc-taxi
Replication 3 set: /datasets/nyc-taxi/yellow_tripdata_2019-02.parquet
Replication 3 set: /datasets/nyc-taxi/yellow_tripdata_2019-03.parquet
Replication 3 set: /datasets/nyc-taxi/yellow_tripdata_2019-04.parquet
Replication 3 set: /datasets/nyc-taxi/yellow_tripdata_2019-05.parquet
Replication 3 set: /datasets/nyc-taxi/yellow_tripdata_2019-06.parquet
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/nyc-taxi
Found 5 items
-rw-r--r--   3 hadoop supergroup  103356025 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-02.parquet
-rw-r--r--   3 hadoop supergroup  116017372 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-03.parquet
-rw-r--r--   3 hadoop supergroup  110139137 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-04.parquet
-rw-r--r--   3 hadoop supergroup  111478943 2026-10-17 13:16 /datasets/nyc-taxi/yellow_tripdata_2019-05.parquet
-rw-r--r--   3 hadoop supergroup  102903344 2026-10-17 13:16 /datasets/nyc-taxi/yellow_tripdata_2019-06.parquet
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks -locations
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&files=1&blocks=1&locations=1&path=%2Fdatasets%2Fnyc-taxi
FSCK started by hadoop (auth:SIMPLE) from /10.244.1.1 for path /datasets/nyc-taxi at Sat Oct 17 13:22:41 UTC 2026

/datasets/nyc-taxi <dir>
/datasets/nyc-taxi/yellow_tripdata_2019-02.parquet 103356025 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742271_1454 len=103356025 Live_repl=3  [DatanodeInfoWithStorage[10.244.1.34:9866,DS-b99c3b1a-af36-4836-9294-
167b119797a4,DISK], DatanodeInfoWithStorage[10.244.2.37:9866,DS-b977b316-376d-4938-9011-fc7bb418f79d,DISK], DatanodeInfoWithStorage[10.244.1.35:9866,DS-0566bc
9b-a9e6-4668-8729-e59f8555614f,DISK]]

/datasets/nyc-taxi/yellow_tripdata_2019-03.parquet 116017372 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742272_1455 len=116017372 Live_repl=3  [DatanodeInfoWithStorage[10.244.2.37:9866,DS-b977b316-376d-4938-9011-
fc7bb418f79d,DISK], DatanodeInfoWithStorage[10.244.1.35:9866,DS-0566bc9b-a9e6-4668-8729-e59f8555614f,DISK], DatanodeInfoWithStorage[10.244.1.34:9866,DS-b99c3b
1a-af36-4836-9294-167b119797a4,DISK]]

/datasets/nyc-taxi/yellow_tripdata_2019-04.parquet 110139137 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742273_1456 len=110139137 Live_repl=3  [DatanodeInfoWithStorage[10.244.1.35:9866,DS-0566bc9b-a9e6-4668-8729-
e59f8555614f,DISK], DatanodeInfoWithStorage[10.244.2.37:9866,DS-b977b316-376d-4938-9011-fc7bb418f79d,DISK], DatanodeInfoWithStorage[10.244.1.34:9866,DS-b99c3b
1a-af36-4836-9294-167b119797a4,DISK]]

/datasets/nyc-taxi/yellow_tripdata_2019-05.parquet 111478943 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742274_1457 len=111478943 Live_repl=3  [DatanodeInfoWithStorage[10.244.2.37:9866,DS-b977b316-376d-4938-9011-
fc7bb418f79d,DISK], DatanodeInfoWithStorage[10.244.1.35:9866,DS-0566bc9b-a9e6-4668-8729-e59f8555614f,DISK], DatanodeInfoWithStorage[10.244.1.34:9866,DS-b99c3b
1a-af36-4836-9294-167b119797a4,DISK]]
```

Figure 2. setrep 3, ls shows replication 3, fsck shows Live_repl=3 on three DataNodes.

The summary at the bottom of the same fsck is the check I was looking for.

- Status: HEALTHY
- Number of data-nodes: 3
- Total files: 5, total size 543894821 bytes (about 519 MB)
- Under-replicated blocks: 0
- Missing replicas: 0
- Corrupt blocks: 0
- Average block replication: 3.0
- Default replication factor: 2 (this is still the cluster default in hdfs-site.xml, not these files)

So the taxi data itself is fully replicated at factor 3. The cluster default stayed 2, which is what new files would get unless I setrep them or change hdfs-site.xml.

```
/datasets/nyc-taxi/yellow_tripdata_2019-06.parquet 102903344 bytes, replicated: replication=3, 1 block(s):  OK
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742275_1458 len=102903344 Live_repl=3  [DatanodeInfoWithStorage[10.244.1.35:9866,DS-0566bc9b-a9e6-4668-8729-
e59f8555614f,DISK], DatanodeInfoWithStorage[10.244.2.37:9866,DS-b977b316-376d-4938-9011-fc7bb418f79d,DISK], DatanodeInfoWithStorage[10.244.1.34:9866,DS-b99c3b
1a-af36-4836-9294-167b119797a4,DISK]]


Status: HEALTHY
 Number of data-nodes:  3
 Number of racks:               1
 Total dirs:                    1
 Total symlinks:                0

Replicated Blocks:
 Total size:    543894821 B
 Total files:   5
 Total blocks (validated):      5 (avg. block size 108778964 B)
 Minimally replicated blocks:   5 (100.0 %)
 Over-replicated blocks:        0 (0.0 %)
 Under-replicated blocks:       0 (0.0 %)
 Mis-replicated blocks:         0 (0.0 %)
 Default replication factor:    2
 Average block replication:     3.0
 Missing blocks:                0
 Corrupt blocks:                0
 Missing replicas:              0 (0.0 %)
 Blocks queued for replication: 0

Erasure Coded Block Groups:
 Total size:    0 B
 Total files:   0
 Total block groups (validated):        0
 Minimally erasure-coded block groups:  0
 Over-erasure-coded block groups:       0
 Under-erasure-coded block groups:      0
 Unsatisfactory placement block groups: 0
 Average block group size:      0.0
 Missing block groups:          0
 Corrupt block groups:          0
 Missing internal blocks:       0
```

Figure 3. fsck summary. HEALTHY, average replication 3.0, under-replicated blocks 0.

3. DataNode failure

The task is to stop one DataNode and write down how long it takes for under-replicated blocks to show up, how HDFS copies the data back, and which commands I used to watch it.

First I tried deleting the pod only:

```
date
kubectl delete pod datanode-2 -n hadoop --wait=false
```

```
[root@hadoop-env ~]# date
Saturday 17 October 2026 08:25:45 AM CDT
[root@hadoop-env ~]# kubectl delete pod datanode-2 -n hadoop --wait=false
pod "datanode-2" deleted from hadoop namespace
[root@hadoop-env ~]#
```

Figure 4. First delete, 08:25:45 AM CDT. The StatefulSet brought the pod back in under a minute.

That did not produce under-replicated blocks. datanode is a StatefulSet, so Kubernetes created datanode-2 again immediately. The NameNode only marks a DataNode dead after it misses heartbeats for about 2 × dfs.namenode.heartbeat.recheck-interval + 10 × dfs.heartbeat.interval. On this cluster those values are 300000 ms and 3 seconds, which is about 10 minutes 30 seconds. The pod was back long before that, on the same disk (same storage id), so the replicas never looked missing.

To keep the node down I scaled the StatefulSet to 2 replicas. That removes datanode-2 and does not recreate it.

```
date
kubectl scale statefulset datanode -n hadoop --replicas=2
kubectl get pods -n hadoop -l app=datanode

# then every couple of minutes
date
kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report | grep -E 'Live datanodes|Dead
datanodes|Under replicated blocks'
kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi | grep -E 'Status:|Number of data-
nodes|Under-replicated|Average block replication|Missing replicas'
```

I scaled at 08:37:36 AM CDT. For the next several minutes fsck still said 3 data-nodes and average replication 3.0. The pod was gone, but the NameNode had not expired it yet. Last contact from datanode-2 was 13:37:49 UTC, which is 08:37:49 AM CDT.

By 13:48:19 UTC (08:48 AM CDT) the report changed to Live datanodes (2) and Dead datanodes (1). That is 10 minutes 30 seconds after the last heartbeat. The “Under replicated blocks: 8” on the cluster report are older small files, not the taxi dataset. I checked the taxi path on its own.

```
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- \
  hdfs dfsadmin -report
Configured Capacity: 197035819008 (183.50 GB)
Present Capacity: 97251158379 (90.57 GB)
DFS Remaining: 95596528466 (89.03 GB)
DFS Used: 1654629913 (1.54 GB)
DFS Used%: 1.70%
Replicated Blocks:
        Under replicated blocks: 8
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (2):
```

Figure 5. After the timeout: Live datanodes (2). The cluster-wide count of 8 under-replicated blocks is older data.

```
[root@hadoop-env ~]# date
Saturday 17 October 2026 08:50:08 AM CDT
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report | grep -E 'Live datanodes|Dead datanodes|Under replicated blocks'
        Under replicated blocks: 8
Live datanodes (2):
Dead datanodes (1):
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi | grep -E 'Status:|Number of data-nodes|Under-replicated|Average block
replication|Missing replicas'
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&path=%2Fdatasets%2Fnyc-taxi
Status: HEALTHY
 Number of data-nodes:  2
 Under-replicated blocks:       0 (0.0 %)
 Average block replication:     3.0
 Missing replicas:              0 (0.0 %)
[root@hadoop-env ~]#
```

Figure 6. 08:50 AM CDT. Dead datanodes (1). Taxi fsck had not dropped the replicas yet, so it still showed average replication 3.0.

A minute later the block map caught up. I ran fsck with -files -blocks and grepped Live_repl. Every taxi block went from Live_repl=3 to Live_repl=2.

- 5 blocks, all Live_repl=2
- Under-replicated blocks: 5 (100%)
- Average block replication: 2.0
- Missing replicas: 5 (33.33%). One missing copy out of three, on five blocks.
- Status stayed HEALTHY. That word only means nothing is corrupt or completely missing. The files are still readable from the two remaining copies.

HDFS did not create a third copy while the node was down. With only 2 live DataNodes it has nowhere to put a third replica. It will not store two copies of the same block on one node. So the blocks stayed under-replicated until datanode-2 came back. If I had 4 DataNodes and RF=3, the NameNode would have copied the missing replica onto the spare node without waiting. If the factor were 4 or 5, those extra copies would also stay missing until I had 4 or 5 live nodes.

```
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks | grep -E 'Live_repl|Status:|Under-replicated|Average bl
ock replication|Missing replicas'
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&files=1&blocks=1&path=%2Fdatasets%2Fnyc-taxi
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742271_1454 len=103356025 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742272_1455 len=116017372 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742273_1456 len=110139137 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742274_1457 len=111478943 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742275_1458 len=102903344 Live_repl=2
Status: HEALTHY
 Under-replicated blocks:       5 (100.0 %)
 Average block replication:     2.0
 Missing replicas:              5 (33.333332 %)
[root@hadoop-env ~]#
```

Figure 7. Failure result. All 5 taxi blocks at Live_repl=2. Under-replicated 100%.

The next screenshot is still the failure. The top of it is the same Live_repl=2 output. At the bottom, at 08:59:01 AM CDT, I scaled the StatefulSet back to 3 so the third copy had a node to return to. The copies are not back yet in this shot.

```
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks | grep -E 'Live_repl|Status:|Under-replicated|Average bl
ock replication|Missing replicas'
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&files=1&blocks=1&path=%2Fdatasets%2Fnyc-taxi
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742271_1454 len=103356025 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742272_1455 len=116017372 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742273_1456 len=110139137 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742274_1457 len=111478943 Live_repl=2
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742275_1458 len=102903344 Live_repl=2
Status: HEALTHY
 Under-replicated blocks:       5 (100.0 %)
 Average block replication:     2.0
 Missing replicas:              5 (33.333332 %)
[root@hadoop-env ~]# date
Saturday 17 October 2026 08:59:01 AM CDT
[root@hadoop-env ~]# kubectl scale statefulset datanode -n hadoop --replicas=3
statefulset.apps/datanode scaled
```

Figure 8. Still Live_repl=2. Bottom of the screen: scale datanode back to 3 at 08:59:01 AM CDT.

How HDFS put the copies back

This is the recovery, after the scale. datanode-2 came back on the same disk (the PVC was still there), so it rejoined as the same DataNode. I waited and ran fsck again.

```
date
kubectl scale statefulset datanode -n hadoop --replicas=3
kubectl get pods -n hadoop -l app=datanode

date
kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks | grep -E
'Live_repl|Under-replicated|Average block replication|Missing replicas'
```

datanode-2 was Running in about 14 seconds. At 09:01:02 AM CDT, about 2 minutes after the scale, fsck showed Live_repl=3 on all 5 blocks, under-replicated blocks 0, average replication 3.0, missing replicas 0. That is the point where the third copy is back.

```
[root@hadoop-env ~]# date
Saturday 17 October 2026 09:01:02 AM CDT
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs fsck /datasets/nyc-taxi -files -blocks | grep -E 'Live_repl|Status:|Under-replicated|Average bl
ock replication|Missing replicas'
Connecting to namenode via http://namenode:9870/fsck?ugi=hadoop&files=1&blocks=1&path=%2Fdatasets%2Fnyc-taxi
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742271_1454 len=103356025 Live_repl=3
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742272_1455 len=116017372 Live_repl=3
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742273_1456 len=110139137 Live_repl=3
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742274_1457 len=111478943 Live_repl=3
0. BP-1105496742-10.244.1.3-1791991909724:blk_1073742275_1458 len=102903344 Live_repl=3
Status: HEALTHY
 Under-replicated blocks:       0 (0.0 %)
 Average block replication:     3.0
 Missing replicas:              0 (0.0 %)
[root@hadoop-env ~]#
```

Figure 9. 09:01:02 AM CDT. This is the recovery. Live_repl=3, under-replicated 0.

Times I would quote for this drill:

- Scaled to 2 DataNodes at 08:37:36 AM CDT.

- NameNode marked datanode-2 dead about 10 minutes 30 seconds later (last heartbeat 08:37:49, dead by 08:48).
- Taxi blocks then showed Live_repl=2 and 5 under-replicated blocks.
- Scaled back to 3 at 08:59:01. Fully replicated again by 09:01:02, about 2 minutes after the pod was back.
- Commands used the whole time: date, kubectl get pods, kubectl scale, hdfs dfsadmin -report, hdfs fsck with -files -blocks.

4. HDFS balancer

The balancer moves blocks from a node that is too full to a node that is too empty. “Too full” is a percent of that node’s disk, not the number of blocks. I used a threshold of 5, which means it only moves data if a node is more than 5% above or below the average percent used.

```
kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report > /tmp/hdfs-report-before-balancer.txt
grep -E 'Live datanodes|Name:|DFS Used%|DFS Used:|Num of Blocks' /tmp/hdfs-report-before-balancer.txt

kubectl exec -n hadoop namenode-0 -- hdfs balancer -threshold 5

kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report > /tmp/hdfs-report-after-balancer.txt
grep -E 'Live datanodes|Name:|DFS Used%|DFS Used:|Num of Blocks' /tmp/hdfs-report-after-balancer.txt
```

Before the run, all three nodes were at 0.84% used. datanode-0 had 527 MB and 238 blocks, datanode-1 had 527.01 MB and 239 blocks, datanode-2 had 523.93 MB and only 14 blocks. datanode-2 has fewer blocks because it was recreated and mostly received the large taxi replicas. The byte counts are almost the same, so the percent used matches.

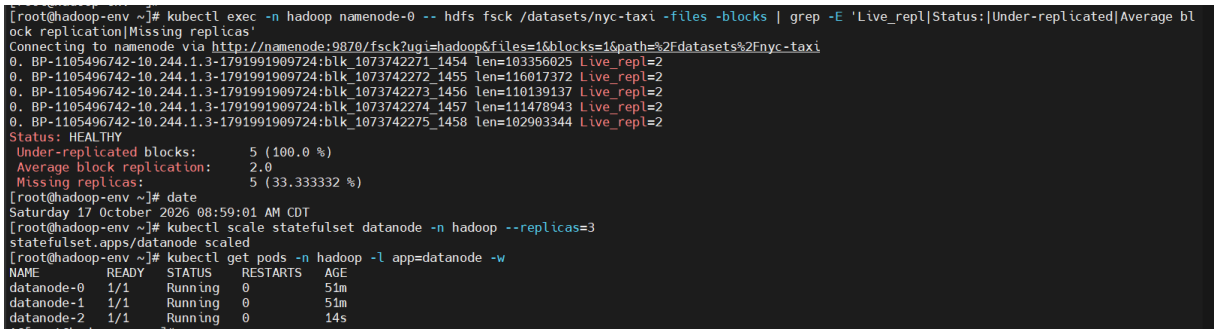


Figure 10. Block distribution before the balancer. All three nodes at 0.84% used.

The balancer ran for 5.903 seconds. It reported 0 over-utilized nodes, 0 under-utilized nodes, Bytes Already Moved 0, Bytes Left To Move 0, and “The cluster is balanced. Exiting.” Nothing needed to move because the disks were already inside the 5% threshold.

```
[root@hadoop-env ~]#
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report > /tmp/hdfs-report-before-balancer.txt
[root@hadoop-env ~]# grep -E 'Live datanodes|Name:|DFS Used%|DFS Used:|Num of Blocks' /tmp/hdfs-report-before-balancer.txt
DFS Used: 1654583296 (1.54 GB)
DFS Used%: 1.70%
Live datanodes (3):
Name: 10.244.1.34:9866 (datanode-1.datanode.hadoop.svc.cluster.local)
DFS Used: 552607744 (527.01 MB)
Non DFS Used: 32813314048 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 239
Name: 10.244.1.40:9866 (datanode-2.datanode.hadoop.svc.cluster.local)
DFS Used: 549376000 (523.93 MB)
Non DFS Used: 32816545792 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 14
Name: 10.244.2.37:9866 (datanode-0.datanode.hadoop.svc.cluster.local)
DFS Used: 552599552 (527 MB)
Non DFS Used: 32813322240 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 238
[root@hadoop-env ~]#
```

Figure 11. Balancer output. Threshold 5. Finished in 5.903 seconds. 0 bytes moved.

The after report is the same picture: three live DataNodes, each about 524–527 MB, each 0.84% used. Block counts stayed 239, 14, and 238. I saved both full reports on the VM as /tmp/hdfs-report-before-balancer.txt and /tmp/hdfs-report-after-balancer.txt.

```
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs balancer -threshold 5
2026-10-17 14:03:47 INFO  Balancer:938 - Using a threshold of 5.0
2026-10-17 14:03:47 INFO  Balancer:738 - namenodes  = [hdfs://namenode:9000]
2026-10-17 14:03:47 INFO  Balancer:739 - parameters = Balancer.BalancerParameters [BalancingPolicy.Node, threshold = 5.0, max idle iteration = 5, #excluded no
des = 0, #included nodes = 0, #source nodes = 0, #blockpools = 0, run during upgrade = false]
2026-10-17 14:03:47 INFO  Balancer:740 - included nodes = []
2026-10-17 14:03:47 INFO  Balancer:741 - excluded nodes = []
2026-10-17 14:03:47 INFO  Balancer:742 - source nodes = []
Time Stamp               Iteration#  Bytes Already Moved  Bytes Left To Move  Bytes Being Moved  NameNode
2026-10-17 14:03:47 INFO  NameNodeConnector:185 - getBlocks calls for hdfs://namenode:9000 will be rate-limited to 20 per second
2026-10-17 14:03:51 INFO  Balancer:264 - dfs.namenode.get-blocks.max-qps = 20 (default=20)
2026-10-17 14:03:51 INFO  Balancer:246 - dfs.balancer.movedWinWidth = 5400000 (default=5400000)
2026-10-17 14:03:51 INFO  Balancer:264 - dfs.balancer.moverThreads = 1000 (default=1000)
2026-10-17 14:03:51 INFO  Balancer:264 - dfs.balancer.dispatcherThreads = 200 (default=200)
2026-10-17 14:03:51 INFO  Balancer:255 - dfs.balancer.getBlocks.size = 2147483648 (default=2147483648)
2026-10-17 14:03:51 INFO  Balancer:255 - dfs.balancer.getBlocks.min-block-size = 10485760 (default=10485760)
2026-10-17 14:03:51 INFO  Balancer:264 - dfs.datanode.balance.max.concurrent.moves = 100 (default=100)
2026-10-17 14:03:51 INFO  Balancer:255 - dfs.datanode.balance.bandwidthPerSec = 104857600 (default=104857600)
2026-10-17 14:03:51 INFO  Balancer:255 - dfs.balancer.max-size-to-move = 10737418240 (default=10737418240)
2026-10-17 14:03:51 INFO  Balancer:255 - dfs.blocksize = 134217728 (default=134217728)
2026-10-17 14:03:51 INFO  NetworkTopology:156 - Adding a new node: /default-rack/10.244.1.34:9866
2026-10-17 14:03:51 INFO  NetworkTopology:156 - Adding a new node: /default-rack/10.244.2.37:9866
2026-10-17 14:03:51 INFO  NetworkTopology:156 - Adding a new node: /default-rack/10.244.1.40:9866
2026-10-17 14:03:51 INFO  Balancer:469 - 0 over-utilized: []
2026-10-17 14:03:51 INFO  Balancer:469 - 0 underutilized: []
Oct 17, 2026 2:03:51 PM          0                 0 B               0 B               0 B                 0  hdfs://namenode:9000
The cluster is balanced. Exiting...
Oct 17, 2026 2:03:51 PM  Balancing took 5.903 seconds
[root@hadoop-env ~]#
[root@hadoop-env ~]#
```

Figure 12. Block distribution after the balancer. Same percent used. No data was moved.

```
[root@hadoop-env ~]#
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -report > /tmp/hdfs-report-after-balancer.txt
[root@hadoop-env ~]# grep -E 'Live datanodes|Name:|DFS Used%|DFS Used:|Num of Blocks' /tmp/hdfs-report-after-balancer.txt
DFS Used: 1654583296 (1.54 GB)
DFS Used%: 1.70%
Live datanodes (3):
Name: 10.244.1.34:9866 (datanode-1.datanode.hadoop.svc.cluster.local)
DFS Used: 552607744 (527.01 MB)
Non DFS Used: 32813359104 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 239
Name: 10.244.1.40:9866 (datanode-2.datanode.hadoop.svc.cluster.local)
DFS Used: 549376000 (523.93 MB)
Non DFS Used: 32816590848 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 14
Name: 10.244.2.37:9866 (datanode-0.datanode.hadoop.svc.cluster.local)
DFS Used: 552599552 (527 MB)
Non DFS Used: 32813367296 (30.56 GB)
DFS Used%: 0.84%
Num of Blocks: 238
[root@hadoop-env ~]#
```

Figure 13. Same after-report, full node lines. datanode-1 239 blocks, datanode-2 14 blocks, datanode-0 238 blocks, all at 0.84%.

5. HDFS snapshots

A snapshot is a read-only view of a directory at one moment. Later changes show up in the live directory. They do not show up inside that snapshot. I enabled snapshots on /datasets, took one named snap-before-change, then added a small test file.

```
kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -allowSnapshot /datasets
kubectl exec -n hadoop namenode-0 -- hdfs dfs -createSnapshot /datasets snap-before-change
kubectl exec -n hadoop namenode-0 -- hdfs lsSnapshottableDir
kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/.snapshot/snap-before-change/nyc-taxi

kubectl exec -n hadoop namenode-0 -- bash -lc 'echo snapshot-test > /tmp/snap-test.txt'
kubectl exec -n hadoop namenode-0 -- hdfs dfs -put -f /tmp/snap-test.txt /datasets/nyc-taxi/snap-test.txt

kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/nyc-taxi
kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/.snapshot/snap-before-change/nyc-taxi
```

allowSnapshot printed “Allowing snapshot on /datasets succeeded”. createSnapshot created /datasets/.snapshot/snap-before-change. Before I added anything, that snapshot already contained the 5 taxi files.

After the put, the live directory /datasets/nyc-taxi had 6 items, including snap-test.txt (14 bytes, 14:19). The snapshot directory still had 5 items, only the parquet files. The new file is not in the snapshot. That is the point of the test. The taxi data was not deleted.

```
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfsadmin -allowSnapshot /datasets
Allowing snapshot on /datasets succeeded
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -createSnapshot /datasets snap-before-change
Created snapshot /datasets/.snapshot/snap-before-change
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs lsSnapshottableDir
drwxr-xr-x 0 hadoop supergroup 0 2026-10-17 14:07 1 65536 /datasets
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/.snapshot/snap-before-change/nyc-taxi
Found 5 items
-rw-r--r--   3 hadoop supergroup  103356025 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-02.parquet
-rw-r--r--   3 hadoop supergroup  116017372 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-03.parquet
-rw-r--r--   3 hadoop supergroup  110139137 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-04.parquet
-rw-r--r--   3 hadoop supergroup  111478943 2026-10-17 13:16 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-05.parquet
-rw-r--r--   3 hadoop supergroup  102903344 2026-10-17 13:16 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-06.parquet
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- bash -lc 'echo snapshot-test > /tmp/snap-test.txt'
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -put -f /tmp/snap-test.txt /datasets/nyc-taxi/snap-test.txt
[root@hadoop-env ~]# echo "===== LIVE DIR ====="
===== LIVE DIR =====
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/nyc-taxi
Found 6 items
-rw-r--r--   2 hadoop supergroup         14 2026-10-17 14:10 /datasets/nyc-taxi/snap-test.txt
-rw-r--r--   3 hadoop supergroup  103356025 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-02.parquet
-rw-r--r--   3 hadoop supergroup  116017372 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-03.parquet
-rw-r--r--   3 hadoop supergroup  110139137 2026-10-17 13:15 /datasets/nyc-taxi/yellow_tripdata_2019-04.parquet
-rw-r--r--   3 hadoop supergroup  111478943 2026-10-17 13:16 /datasets/nyc-taxi/yellow_tripdata_2019-05.parquet
-rw-r--r--   3 hadoop supergroup  102903344 2026-10-17 13:16 /datasets/nyc-taxi/yellow_tripdata_2019-06.parquet
[root@hadoop-env ~]# echo "===== SNAPSHOT DIR ====="
===== SNAPSHOT DIR =====
[root@hadoop-env ~]# kubectl exec -n hadoop namenode-0 -- hdfs dfs -ls /datasets/.snapshot/snap-before-change/nyc-taxi
Found 5 items
-rw-r--r--   3 hadoop supergroup  103356025 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-02.parquet
-rw-r--r--   3 hadoop supergroup  116017372 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-03.parquet
-rw-r--r--   3 hadoop supergroup  110139137 2026-10-17 13:15 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-04.parquet
-rw-r--r--   3 hadoop supergroup  111478943 2026-10-17 13:16 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-05.parquet
-rw-r--r--   3 hadoop supergroup  102903344 2026-10-17 13:16 /datasets/.snapshot/snap-before-change/nyc-taxi/yellow_tripdata_2019-06.parquet
[root@hadoop-env ~]#
```

Figure 14. Snapshot snap-before-change has 5 files. The live directory has those 5 plus snap-test.txt.

What this part showed

- Uploaded 5 NYC taxi parquet files, 518.7 MB, to /datasets/nyc-taxi.
- Part 1 default is RF=2 because there were 2 DataNodes. I added a 3rd DataNode and set this dataset to RF=3.
- fsck confirmed Live_repl=3, average replication 3.0, under-replicated blocks 0. The cluster default line still says 2.

- RF=4 would need 4 DataNodes. RF=5 would need 5. Extra copies stay missing if there are not enough nodes.
- Stopping datanode-2 for real meant scaling the StatefulSet to 2. The NameNode waited about 10 minutes 30 seconds, then the 5 taxi blocks became under-replicated at Live_repl=2.
- HDFS could not re-replicate until the 3rd node returned. After I scaled back to 3, the copies were restored by 09:01:02, about 2 minutes later.
- The balancer compared percent used, found all nodes at 0.84%, moved 0 bytes, and exited in 5.9 seconds. Before and after reports match.
- A snapshot of /datasets kept the old listing. A file added afterwards showed up only in the live directory.